

Plan — Puesta en producción del microservicio en Azure

FGF Trapani · Reporte VENTAS INDUSTRIA · Microservicio Finnegans → Excel / Power BI

Fecha: 2026-07 · Autor: Agustín (con asistencia de Claude)

La meta

Que el servicio viva en internet (en Azure), que **todas las noches a las 3am se traiga solo** los datos de Finnegans y los deje listos, y que el Excel/Power BI los pida al instante, **sin que nadie tenga que hacer nada a mano** (ni pegar tokens ni apretar botones).

Las piezas que vamos a usar (y para qué sirve cada una)

#	Pieza	Para qué sirve
1	Azure App Service (Linux, Python)	El “servidor” administrado donde corre la app. Azure maneja el sistema, los reinicios y el HTTPS. Plan robusto (Premium) con “Always On” prendido para que el proceso nunca se apague . Una sola instancia.
2	Manejo de token propio	Hoy el token lo pega el Excel y se vence. El servicio va a generar su propio token y renovarlo solo . Saca al Excel del problema del token. Usa credenciales de la empresa (no las de Marco).
3	Programador 3am (APScheduler)	Pieza chica de Python que a las 3am llama a “traé todo” y deja el cache caliente . Va dentro del mismo servicio, no es tecnología aparte.
4	Azure Cache for Redis	Guarda el resultado ya traído. A diferencia del cache “en memoria”, sobrevive reinicios y actualizaciones . El Excel siempre encuentra el dato.
5	Azure Key Vault	Un “cofre” para guardar las credenciales fuera del código . Nada de claves escritas en los archivos.
6	GitHub Actions (CI/CD)	Conecta GitHub con Azure: cada cambio que subís se publica solo . Sin copiar archivos a mano.
7	Application Insights	El “tablero de salud”: guarda los logs y avisa si el job de las 3am falla o si algo anda mal. Te enterás vos antes que Marco.
8	Seguridad (candado)	Que solo el Excel/Power BI entren, con una clave de acceso (API key) o lista de IPs permitidas. Que no quede abierto a internet.

Orden de trabajo (por etapas)

Etapas	Qué hacemos	Resultado
0. Preparar el repo	Dejar la app lista para Azure (arranque, dependencias, config por variables de entorno).	Repo “deployable”.
1. Token propio	Que el servicio genere y renueve su token solo.	Ya no depende del Excel para el token.
2. Reloj 3am	Agregar el programador + un endpoint /refresh manual por las dudas.	Refresco automático nocturno.
3. Redis	Mover el cache a Redis.	El dato sobrevive reinicios y deploys.
4. Infra en Azure	Crear App Service + Redis + Key Vault + App Insights.	Todo el entorno armado.

5. CI/CD	Conectar GitHub → deploy automático.	Publicar con solo subir código.
6. Seguridad + pruebas	Poner la clave de acceso, probar de punta a punta, activar alertas.	Listo para que lo use Marco.

Decisiones tomadas (las más convenientes)

- **App Service** en vez de Container Apps/Functions → lo más directo para esto.
- **Redis** en vez de cache en memoria → para que el dato no se pierda nunca.
- **Programador adentro de la app** en vez de disparador externo → menos piezas, más simple.

Lo que se necesita para arrancar

1. **Credenciales de la empresa** (client_id / client_secret) para que el servicio genere su token — las de Agustín o las que le habiliten, **no las de Marco**.
2. Acceso a la **cuenta de Azure** donde se crea todo (o que IT lo habilite).

Primer paso concreto: **Etapas 1 (token propio)**, porque sin eso el 3am no puede autenticarse y además saca el token vencido del Excel.